

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	Pranitha	Reg. No.:	192472073
Section / Batch:	CSE AI	Date:	16 july 2026

Code of Conduct

I am Pranitha 192472073 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: pranitha

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.

Identify email addresses.

Extract mobile numbers.

Detect technical skills (Python, Java, SQL, Machine Learning, NLP).

Extract years of experience.

Generate a structured summary of the candidate's profile.

Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

```
import re

resumes = [
    """Name: John
    Email: john@gmail.com
    Phone: 9876543210
    Skills: Python, SQL
    Experience: 3 years""",
    """Name: Priya
    Email: priya@gmail.com
    Phone: 9123456780
    Skills: Java
    Experience: 1 year""",
]

for r in resumes:
    name = re.search(r"Name:\s*(.*)", r).group(1)
    email = re.search(r"[\w\.-]+@[\w\.-]+\.\w+", r).group()
    phone = re.search(r"[6-9]\d{9}", r).group()
    skills = re.findall(r"Python|Java|SQL|Machine Learning|NLP", r)
    exp = int(re.search(r"\d+", re.search(r"Experience:.*", r).group()).group())

    print("Name:", name)
    print("Email:", email)
    print("Phone:", phone)
    print("Skills:", ", ".join(skills))
    print("Experience:", exp, "years")

    if exp >= 2 and "Python" in skills:
        print("Eligible Candidate")
    else:
        print("Not Eligible")
    print()
```

OUTPUT

```
▼ ... Name: John
      Email: john@gmail.com
      Phone: 9876543210
      Skills: Python, SQL
      Experience: 3 years
      Eligible Candidate

      Name: Priya
      Email: priya@gmail.com
      Phone: 9123456780
      Skills: Java
      Experience: 1 years
      Not Eligible
```

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.
Perform prefix-based searches.
Perform suffix-based searches.
Search using partial keywords.
Perform case-insensitive searches.
Display all matching product names.
Generate a report showing the total number of matching products for each search.

```
import re

products = ["Laptop", "Lapdesk", "Desktop", "Mouse", "Keyboard", "Gaming Laptop"]

searches = {
    "Exact": r"^Laptop$",
    "Prefix": r"^Lap",
    "Suffix": r"top$",
    "Partial": r"top",
    "Case-Insensitive": r"laptop"
}

for name, pattern in searches.items():
    result = [p for p in products if re.search(pattern, p, re.IGNORECASE)]
    print(name, "Search:", result)
    print("Total Matches:", len(result))
    print()
```

```

✓ ... Exact Search: ['Laptop']
    Total Matches: 1

    Prefix Search: ['Laptop', 'Lapdesk']
    Total Matches: 2

    Suffix Search: ['Laptop', 'Desktop', 'Gaming Laptop']
    Total Matches: 3

    Partial Search: ['Laptop', 'Desktop', 'Gaming Laptop']
    Total Matches: 3

    Case-Insensitive Search: ['Laptop', 'Gaming Laptop']
    Total Matches: 2

```

OUTPUT

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

- Validate the student register number.
- Validate the institutional email address.
- Validate the course code.
- Validate the semester information.
- Validate the student's mobile number.
- Display appropriate validation messages for each field.
- Generate a final registration status report indicating whether the registration is successful.

```

import re

regno = "23CS101"
email = "student@university.edu"
course = "CS301"
semester = "5"
mobile = "9876543210"

print("Register No.:", "Valid" if re.fullmatch(r"\d{2}[A-Z]{2}\d{3}", regno) else "Invalid")
print("Email:", "Valid" if re.fullmatch(r"\w+@university\.edu", email) else "Invalid")
print("Course Code:", "Valid" if re.fullmatch(r"[A-Z]{2}\d{3}", course) else "Invalid")
print("Semester:", "Valid" if re.fullmatch(r"[1-8]", semester) else "Invalid")
print("Mobile:", "Valid" if re.fullmatch(r"[6-9]\d{9}", mobile) else "Invalid")

if (re.fullmatch(r"\d{2}[A-Z]{2}\d{3}", regno) and
    re.fullmatch(r"\w+@university\.edu", email) and
    re.fullmatch(r"[A-Z]{2}\d{3}", course) and
    re.fullmatch(r"[1-8]", semester) and
    re.fullmatch(r"[6-9]\d{9}", mobile)):
    print("\nRegistration Status: Successful")
else:
    print("\nRegistration Status: Failed")

```

```
✓ ... Register No.: Valid  
    Email: Valid  
    Course Code: Valid  
    Semester: Valid  
    Mobile: Valid  
  
    Registration Status: Successful
```

OUTPUT

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____